

TECNM CAMPUS CELAYA

VYTARIS

Sistema de Perfil Médico de Emergencia

Versión 1.0 · 2026

Documentación Técnica

Arquitectura · APIs · Flujo de Datos · Despliegue

Este documento describe la arquitectura, componentes y funcionamiento interno de la aplicación web Vytaris. Está dirigido a desarrolladores, evaluadores y cualquier persona interesada en comprender el sistema en profundidad.

1. Introducción

Vytaris es una aplicación web de página única (SPA) diseñada para generar perfiles médicos de emergencia codificados en códigos QR. Su propósito es que cualquier persona pueda registrar su información médica crítica — tipo de sangre, alergias, contacto de emergencia— y obtener un código QR que puede imprimirse en un llavero o pulsera.

Cuando un paramédico o cualquier persona escanea ese QR, el navegador abre inmediatamente la tarjeta de emergencia con todos los datos de la persona, sin necesidad de instalar ninguna aplicación adicional.

Características principales:

- **100% del lado del cliente** — no existe ningún servidor, base de datos ni registro de usuarios.
- **Privacidad total** — los datos nunca salen del dispositivo; viven exclusivamente dentro del QR impreso.
- **Sin instalación** — funciona en cualquier navegador moderno con solo escanear el QR.
- **Mobile-first** — diseñado prioritariamente para leerse en smartphones, pensando en el paramédico en campo.
- **Código abierto y auditable** — tres archivos de texto plano, sin dependencias de build.

2. Estructura del Proyecto

El proyecto es intencionalmente minimalista. No requiere proceso de compilación, empaquetado ni herramientas de build. Todos los archivos se sirven directamente como archivos estáticos.

Archivo	Tipo	Responsabilidad
index.html	HTML	Esqueleto visual de la app: define la estructura de ambas pantallas, los íconos SVG y los formularios.
style.css	CSS	Toda la apariencia visual: colores, tipografía, animaciones y adaptación a distintos tamaños de pantalla.
script.js	JS	Toda la lógica de la aplicación: validación, codificación de datos, generación del QR y renderizado de la tarjeta de emergencia.
logo.png	PNG	Logotipo oficial de Vytaris en formato PNG con fondo transparente. Se muestra en el encabezado de la página.

Principio de diseño: separación de responsabilidades

Cada archivo tiene una única responsabilidad y no depende de herramientas externas de build. Esta decisión facilita el mantenimiento, la auditoría de código y el despliegue en cualquier hosting de archivos estáticos sin configuración adicional.

3. Arquitectura de la Aplicación

3.1 Arquitectura sin servidor (Serverless Static)

Vytaris adopta una arquitectura radicalmente simple: no existe backend. No hay base de datos, no hay API REST, no hay sesiones de usuario ni ningún procesamiento del lado del servidor. La aplicación completa vive en el navegador del usuario.

USUARIO	NAVEGADOR WEB	VERCEL CDN (archivos estáticos)
Escanea QR / Llena formulario	Ejecuta JS · Renderiza UI	index.html · style.css · script.js

3.2 Enrutamiento del lado del cliente

La aplicación tiene dos modos de operación que se determinan al cargar la página inspeccionando los parámetros de la URL:

- **Modo Formulario** — URL sin parámetros. Se muestra el formulario de captura de datos.
- **Modo Tarjeta de Emergencia** — URL con el parámetro `?v_info=...` Se decodifican los datos y se muestra la tarjeta médica.

Esta técnica elimina la necesidad de un servidor de rutas. Un solo archivo HTML sirve ambos modos.

4. APIs y Librerías Utilizadas

4.1 QRCode.js — Generación del código QR

Es la única dependencia externa del proyecto. Se carga directamente desde un CDN (red de distribución de contenido) sin necesidad de instalación ni proceso de compilación.

```
https://cdn.jsdelivr.net/gh/davidshimjs/qrcodejs@gh-pages/qrcode.min.js
```

Aspecto	Detalle
Autor	Shim Sangmin (davidshimjs)
Licencia	MIT — uso libre incluyendo proyectos comerciales
API utilizada	<code>new QRCode(elemento, opciones)</code> — síncrona, crea un <code><canvas></code> directamente en el DOM
Por qué esta lib	Funciona en el navegador sin bundler ni Node.js. Compatible con Vercel estático.
Nivel de corrección	<code>QRCode.CorrectLevel.H</code> (alto) — el QR sigue siendo legible aunque esté parcialmente dañado

4.2 Web APIs nativas del navegador

El resto de la funcionalidad se construye exclusivamente sobre APIs estándar que cualquier navegador moderno ofrece de forma nativa, sin importar librerías adicionales.

TextEncoder / TextDecoder

Convierte texto con caracteres especiales (tildes, ñ) a bytes y viceversa. Garantiza que los datos se codifiquen correctamente en Base64 independientemente del idioma.

btoa() / atob()

Funciones nativas para codificar y decodificar Base64. Convierten los datos a un formato seguro para incluirse en una URL.

URLSearchParams

Lee los parámetros de la URL de forma segura. Se usa para detectar si la URL contiene datos y extraerlos.

URL API / window.location

Permite construir la URL completa del perfil que se codificará dentro del QR, adaptándose automáticamente al dominio donde esté desplegada la app.

DOM API

La interfaz estándar para manipular el HTML desde JavaScript: crear elementos, cambiar texto, mostrar u ocultar secciones, agregar eventos a botones, etc.

Canvas API

La librería QRCode.js utiliza internamente un <canvas> para dibujar el QR como imagen. También se usa para exportar el QR como archivo PNG descargable.

HTMLAnchorElement (download)

Permite desencadenar la descarga de un archivo desde JavaScript creando un enlace invisible con el atributo download y llamando a .click() programáticamente.

5. Manejo de Datos

5.1 Ciclo completo del dato — de formulario a QR

Los datos nunca se almacenan en un servidor. El único "almacenamiento" es el propio código QR impreso en físico. A continuación se describe cada paso del proceso:

1. Captura

El usuario llena el formulario HTML. Los valores se leen desde los elementos del DOM.

2. Validación

Se verifica que los campos obligatorios estén completos y que la fecha de nacimiento sea válida. Si hay errores, se muestran mensajes bajo cada campo.

3. Serialización JSON

Los datos del formulario se convierten a un objeto JavaScript y luego a texto JSON: `{"nombre": "...", "sangre": "O+", ...}`

4. Codificación UTF-8

`TextEncoder` convierte el texto JSON a bytes, garantizando soporte para tildes, ñ y cualquier carácter Unicode.

5. Codificación Base64

Los bytes se convierten a una cadena Base64 con `btoa()`. Resultado: texto compuesto solo por letras, números y `+/=`, seguro para URLs.

6. Construcción de URL

Se arma la URL final: `https://vytaris.vercel.app/?v_info=<base64>`. `encodeURIComponent()` limpia los caracteres especiales del Base64 para la URL.

7. Generación del QR

La librería `QRCode.js` recibe la URL completa y dibuja el código QR en un elemento `<canvas>` del HTML.

8. Descarga

El Canvas se exporta como imagen PNG y se ofrece al usuario para descargar e imprimir.

5.2 Ciclo inverso — de QR a tarjeta de emergencia

Cuando alguien escanea el QR, el teléfono abre la URL. La aplicación detecta el parámetro `v_info` y revierte el proceso:

1. Lectura de URL

`URLSearchParams.get("v_info")` extrae el valor Base64 de la URL.

2. Decodificación Base64

`atob()` convierte el Base64 de vuelta a bytes binarios.

3. Decodificación UTF-8

`TextDecoder` convierte los bytes al texto JSON original, recuperando tildes y caracteres especiales.

4. Parsing JSON

JSON.parse() reconstruye el objeto JavaScript con todos los datos de la persona.

5. Renderizado

La aplicación construye dinámicamente el HTML de la tarjeta médica con los datos obtenidos y lo muestra en pantalla.

6. Cálculo de edad

La edad se calcula en tiempo real a partir de la fecha de nacimiento almacenada, siempre actualizada aunque el QR tenga años.

5.3 Estructura del objeto de datos

El objeto JSON que viaja dentro del QR tiene la siguiente estructura:

```
{
  "nombre":      "María García Pérez",
  "nacimiento":  "1985-03-12",
  "postal":      "38020",
  "contacto":    "Mamá — 55 1234 5678",
  "sangre":      "O+",
  "alergias":    "Penicilina, látex",
  "condiciones": "Diabetes tipo 2"
}
```

6. Componentes Principales

6.1 Encabezado (Header)

Barra azul fija en la parte superior, siempre visible independientemente del scroll. Contiene el logotipo y la frase de marca. Implementada con CSS position: sticky; top: 0.

6.2 Modo Formulario — Captura de datos

Pantalla principal que aparece cuando la URL no contiene datos. Campos disponibles:

- **Nombre completo** — Obligatorio. Texto libre.
- **Fecha de nacimiento** — Obligatorio. Selector de fecha nativo. Se bloquean fechas futuras.
- **Código postal** — Opcional. Solo acepta dígitos (filtrado en tiempo real).
- **Contacto de emergencia** — Obligatorio. Texto libre; el sistema extrae el número para el botón de llamada.
- **Tipo de sangre** — Obligatorio. Lista con todos los tipos (A+, A-, B+, B-, AB+, AB-, O+, O-, Desconocido).
- **Alergias conocidas** — Opcional. Área de texto libre.
- **Enfermedades crónicas** — Opcional. Área de texto libre.

Validación de formulario

Antes de generar el QR se validan todos los campos obligatorios. Si alguno falla, se muestra un mensaje de error bajo el campo, el campo se marca en rojo y el cursor se mueve automáticamente al primer campo con error. Los errores desaparecen en cuanto el usuario empieza a corregir el campo.

6.3 Panel del QR generado

Aparece bajo el formulario tras generar exitosamente el QR. Contiene:

- El código QR dibujado en un <canvas> de 256×256 px.
- Una vista previa truncada de la URL codificada dentro del QR.
- Botón de descarga: exporta el canvas como imagen PNG.
- Botón "Crear otro perfil": limpia el formulario y oculta el panel.

6.4 Modo Tarjeta de Emergencia — Lectura

Pantalla que reemplaza al formulario cuando se detecta el parámetro v_info en la URL. Diseñada para que un paramédico lea la información crítica de un vistazo:

- Banner rojo con animación de brillo — atrae la atención inmediatamente.
- Nombre, edad calculada en tiempo real y tipo de sangre en badge prominente.
- Alergias y enfermedades con fondo naranja de advertencia.
- Contacto de emergencia y botón de llamada directa (href="tel:...").
- Aviso legal de privacidad al pie.

7. Flujo Completo de la Aplicación

Paso	Acción	Descripción técnica
Paso 1	El usuario abre la app	Accede a la URL sin parámetros. <code>m_iniciar()</code> detecta que no hay <code>v_info</code> y muestra el formulario.
Paso 2	Llena sus datos	Completa los campos. Los errores se muestran/ocultan en tiempo real conforme escribe.
Paso 3	Genera el QR	<code>m_validarFormulario()</code> verifica los campos. <code>m_codificar()</code> convierte los datos a Base64. <code>m_construirUrl()</code> arma la URL. <code>QRCode</code> dibuja el código en el canvas.
Paso 4	Descarga e imprime	Presiona "Descargar imagen". El canvas se exporta como PNG. El usuario imprime el QR y lo coloca en su llavero.
Paso 5	Escaneo de emergencia	Un paramédico escanea el QR con cualquier cámara. El teléfono abre la URL con el parámetro <code>v_info</code> .
Paso 6	Decodificación	<code>m_iniciar()</code> detecta <code>v_info</code> . <code>m_decodificar()</code> recupera los datos. <code>m_mostrarTarjetaEmergencia()</code> renderiza la tarjeta.
Paso 7	Acción de emergencia	El paramédico lee los datos. Si hay número de teléfono, puede llamar directamente con un toque.

8. Privacidad y Seguridad

8.1 Modelo de privacidad

Vytaris fue diseñado desde su núcleo con privacidad por defecto. El modelo es radicalmente diferente al de la mayoría de aplicaciones médicas:

- **Sin servidor** — Los datos nunca viajan a ningún servidor.
- **Sin base de datos** — No existe ninguna tabla, colección o archivo donde se guarden datos de usuarios.
- **Sin cookies ni localStorage** — La app no utiliza almacenamiento persistente en el navegador.
- **Solo en el QR físico** — Los datos existen únicamente en el código impreso. Si el usuario destruye el QR, los datos desaparecen.
- **Voluntario** — El usuario elige qué información incluir.

8.2 Prevención de XSS (Cross-Site Scripting)

Toda información proveniente del QR se sanea antes de insertarse en el HTML. Se aprovecha el propio navegador para escapar caracteres peligrosos (<, >, &, etc.) sin depender de ninguna librería externa de sanitización:

```
const m_limpiarTexto = (texto) => {
  const div = document.createElement("div");
  div.textContent = texto; // texto plano, NO HTML
  return div.innerHTML;   // recuperar ya escapado
};
```

8.3 Integridad de datos en el QR

La codificación Base64 no es cifrado — es solo una transformación de formato. Los datos son legibles por cualquiera que decodifique el Base64. Esta es una decisión de diseño deliberada: en una emergencia, los paramédicos necesitan acceder a los datos con cualquier herramienta disponible, sin depender de claves o cuentas.

9. Diseño y Experiencia de Usuario

9.1 Enfoque Mobile-first

El CSS se escribe primero para pantallas pequeñas y se ajusta progresivamente para pantallas más grandes mediante media queries.

Breakpoint	Descripción
Base (< 480px)	Móviles pequeños — columna única, tipografía optimizada para una mano
480px	Teléfonos medianos — logo más grande, más padding en las tarjetas
640px	Tablets y teléfonos grandes — tipografía ligeramente mayor
900px	Escritorio — eslogan a tamaño completo

9.2 Paleta de colores

Hex	Nombre	Uso
#1A56B2	Azul principal	Header, botón principal, íconos del formulario
#2DC78A	Verde acento	Botón de descarga, franja inferior
#C0392B	Rojo emergencia	Banner de alerta, encabezado de tarjeta médica, errores
#D35400	Naranja alerta	Campos de alergias y enfermedades en la tarjeta
#EDF2F7	Gris fondo	Fondo de la página y de los valores en la tarjeta

9.3 Animaciones

- **pulso-brillo** — El banner rojo de emergencia pulsa suavemente para llamar la atención al escanear el QR.
- **pop-entrada** — El badge del tipo de sangre aparece con un efecto de "rebote" al cargar la tarjeta.

10. Despliegue

10.1 Plataforma — Vercel

La aplicación está desplegada en Vercel, una plataforma de hosting especializada en sitios estáticos. Vercel sirve los archivos desde su CDN global, garantizando baja latencia en cualquier parte del mundo.

- **URL pública:** <https://vytaris.vercel.app>
- **Tipo de despliegue:** Archivos estáticos — sin servidor, sin contenedor, sin configuración de runtime.
- **Trigger:** Cada push a la rama main del repositorio GitHub desencadena automáticamente un nuevo deploy.
- **HTTPS:** Certificado SSL/TLS gestionado automáticamente por Vercel.

10.2 Requisitos para desplegar en cualquier hosting

Al no tener backend, Vytaris puede desplegarse en cualquier servicio que sirva archivos estáticos:

- GitHub Pages, Netlify, Cloudflare Pages, Firebase Hosting, AWS S3 + CloudFront.
- Cualquier servidor web convencional (Apache, Nginx) con acceso directo a los archivos.
- No se requiere soporte de PHP, Python, Node.js ni ningún runtime del lado del servidor.
- No se requiere base de datos ni variables de entorno.

10.3 Control de versiones

El proyecto se gestiona con Git y se aloja en GitHub. Cada versión significativa se etiqueta con un commit de versión (v1, v2... v8 en la versión actual). La integración continua de Vercel despliega automáticamente cada nuevo commit en la rama principal.

11. Convenciones de Código

El archivo script.js sigue una convención de nomenclatura consistente que permite identificar el tipo de cada variable o función con solo ver su nombre:

Prefijo	Tipo	Ejemplo	Descripción
k_	Constante / referencia DOM	k_inputName	Valor que no cambia. Normalmente un elemento del HTML.
v_	Variable local	v_datos	Valor que puede cambiar dentro de una función o bloque.
p_	Parámetro de función	p_fechaISO	Valor que se recibe como argumento al llamar una función.
m_	Función (método)	m_generarQR	Bloque de código que realiza una tarea específica.

Adicionalmente, todo el código utiliza "use strict" para activar el modo estricto de JavaScript, que detecta errores comunes y previene comportamientos inesperados. Los comentarios están escritos en español con lenguaje accesible para que cualquier persona con nociones básicas de programación pueda entender el código.

12. Conclusión

Vytaris demuestra que es posible construir una herramienta de salud útil, privada y accesible sin recurrir a infraestructura compleja. Al eliminar el servidor de la ecuación, se eliminan también los riesgos de filtración de datos médicos, los costos de mantenimiento de backend y la dependencia de servicios externos que pueden desaparecer.

La app puede escanearse en cualquier parte del mundo, funciona sin conexión a internet una vez cargada la primera vez, y es tan fácil de actualizar como editar tres archivos de texto y hacer push a GitHub.

Característica	Decisión de diseño
Privacidad	Sin servidor — datos solo en el QR impreso
Accesibilidad	Sin app — funciona con cualquier cámara y navegador
Mantenimiento	Sin backend — sin servidores que mantener ni actualizar
Despliegue	Archivos estáticos — compatible con cualquier CDN o hosting
Extensibilidad	Código simple y bien comentado — fácil de modificar o bifurcar
Rendimiento	Una sola librería externa (< 30 KB) — carga instantánea